

claude-windows / c6028e72-ebfd-4218-89a8-c4d7dd23be96

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c6028e72-ebfd-4218-89a8-c4d7dd23be96\subagents\workflows\wf_d251ad6c-472\agent-a41002c8e591fad0c.jsonl`  
- SHA-256: `bccd16c96c119ad0a167476074a6b74d14cdd6f2650b5699dfcbad60feb82207`  
- Source modified: `2026-06-08T08:45:36+00:00`  
- Imported at: `2026-07-05T16:48:23+00:00`  
- project: `wf_d251ad6c-472`  
- session_id: `c6028e72-ebfd-4218-89a8-c4d7dd23be96`
```

Transcript

1. user (2026-06-08T08:43:41.063Z)

LENS = LABEL-LIFECYCLE / PROVENANCE PRODUCT. Propose positioning the collector around the LABEL LIFECYCLE: provenance + licensing per source, partial-label maturity (needs_review -> confirmed -> gold), the labeled-window concept (a video tagged only to t=T), and the golden-set regression flow so re-tagging improves a tracked corpus. Emphasize how the collector records WHAT is trustworthy and HOW MUCH of each video is labeled, so the indexer eval restricts to labeled windows automatically.

Ground every recommendation in these subsystem FINDINGS (cite files). Do not invent files or fields that the findings do not support.

FINDINGS:

```
[  
  {  
    "subsystem": "Sports Data Collector - Local/Manual Ingestion CLI",  
    "summary": "The sports-data-collector provides a manual ingestion pipeline (`import-local` command) that accepts local video files paired with JSON or CSV annotation files in a sport-specific rally/point/delivery format. It parses annotations, validates temporal consistency (no overlaps, ordered progression), registers metadata + segments in SQLite, and exports curated data as indexer-ready eval CSVs containing only (start, end) decimal-second intervals per video.",  
    "key_facts": [  
      "CLI entry: src/cli/curate.py:844-961 (main function with subparser setup); import-local command registered line 854-863",  
      "import_local() method: src/cli/curate.py:52-155; accepts --sport (required, lowercase), --video-path (required), --annotations-path (required), --match-name (optional, defaults to video basename), --video-id (optional, defaults to lowercased match_name), --license (default='Proprietary'), --fps (default=30.0), --resolution (default='1080p')",  
      "_parse_local_annotations() method: src/cli/curate.py:156-302; accepts .json or .csv files; normalizes CSV headers to lowercase; raises ValueError if unsupported extension",  
      "Badminton schema: src/core/schemas/badminton.py:4-22; required fields: video_id, rally_number (int), start_time (float), end_time (float); optional: score_before, score_after, shots_count (int), ending_reason, last_shot_outcome; validates end_time > start_time",  
      "CSV parsing: src/cli/curate.py:232-298; uses csv.DictReader with normalized lowercase headers; safe_int() with idx+1 default for rally_number; safe_float_required() for start_time, end_time (raises ValueError if missing/empty); gracefully degrades optional fields to None",  
      "JSON parsing: src/cli/curate.py:161-231; expects list of objects; uses identical safe_int/safe_float logic; treats missing required fields (start_time, end_time) as ValueError",  
      "Validation pipeline: src/cli/curate.py:88-106 calls validator.validate_badminton_rallies() at src/core/validator.py:29-57; checks: no zero/negative duration (start_time < end_time), no temporal overlaps between sequential rallies (sorted by rally_number), chronological progression; returns (bool, List[str]) errors",  
      "DB insertion: src/cli/curate.py:123-136 inserts VideoMetadata (status=CURATED for manual imports, bypassing staging) + rallies; db.insert_badminton_rallies() at src/core/db.py:210-237 performs UPSERT (DELETE then INSERT per video_id)",  
    ]  
  }  
]
```

```

    "Export format: export_eval_set() at src/cli/curate.py:440-536; emits per-video CSV with
header [start,end] + decimal-second tuples (queried via get_segment_intervals() at
src/core/db.py:527-546); manifest.json pairs CSVs with video metadata (local_path, video_url,
fps, resolution, etc.)",
    "License handling: LicenseType enum at src/core/schemas/base.py:12-22 (MIT, Apache-2.0,
BSD, CC0, CC-BY, CC-BY-NC, CC-BY-NC-SA, Public-Domain, Proprietary, Unknown);
validator.check_license_commercial() at src/core/validator.py:24-27 checks against whitelist
(config default: MIT, Apache-2.0, BSD, CC0, CC-BY, Public-Domain, Proprietary)",
    "CurationStatus enum: src/core/schemas/base.py:24-27 (STAGING, CURATED, REJECTED); manual
imports set status=CURATED directly (line 117)",
    "Rally-annotator CSV format compatibility: docs/CSV_FORMAT.md specifies columns
(rally_number, start_time, end_time, ending_reason, sport); decimal seconds, 1-based rally
numbers; sports-data-collector import-local reads this directly (compatible via DictReader
header matching)"
],
"contracts": [
    "import-local CLI flags: --sport (required:
badminton|tennis|cricket|pickleball|tabletennis), --video-path (required: file path),
--annotations-path (required: .json or .csv), --match-name (str, optional), --video-id (str,
optional), --license (str, default='Proprietary'), --fps (float, default=30.0), --resolution
(str, default='1080p')",
    "CSV input columns (required): rally_number (int or auto-indexed), start_time (float,
decimal seconds), end_time (float, decimal seconds); optional: score_before, score_after,
shots_count, ending_reason, last_shot_outcome. Case-insensitive header matching via lowercase
normalization.",
    "JSON input structure: list of objects, each with keys: rally_number (int), start_time
(float), end_time (float), plus optional fields matching CSV",
    "Badminton rally object contract: video_id (str), rally_number (int), start_time (float),
end_time (float); start_time must be < end_time; rally_number serves as chronological sort key",
    "DB schema: badminton_rallies table (video_id, rally_number, start_time, end_time,
score_before, score_after, shots_count, ending_reason, last_shot_outcome) with PK=(video_id,
rally_number), FK on videos.video_id with ON DELETE CASCADE",
    "Export format: per-video CSV at <out_dir>/<sport>/<video_id>.csv with header [start,end]
and (start_time, end_time) decimal-second tuples in chronological order; manifest.json with
entries {video_id, sport, csv, local_path, video_url, match_name, fps, resolution, license_type,
is_commercial, status, num_segments}"
],
"gaps_or_mismatches": [
    "Fractional rally_number support: rally-annotator can produce decimal rally_number (e.g.,
2.5 for resegmented rallies); sports-data-collector expects int, converts via safe_int() which
truncates (2.5 -> 2), causing loss of distinction between fractional variants; no explicit error
on truncation",
    "Partial coverage handling: rally-annotator CSV may have gaps (e.g., rally 1,2,4 missing
3) or float numbers (1.1, 1.2); validator checks only temporal non-overlap and chronological
order via rally_number sort, not contiguity or valid numbering schemes",
    "Missing optional fields: sport column in rally-annotator CSV is self-documenting but
ignored by import-local; no validation that CSV sport matches --sport flag",
    "Score/outcome fields: rally-annotator tracks ending_reason enum
(winner|forced_error|unforced_error|service_fault|let|other); maps exactly to
BadmintonRally.ending_reason (optional); no score_before/score_after in rally-annotator, must be
None in import",
    "Null/empty handling: safe_float_required() treats empty string as ValueError; optional
int fields default to None (not 0) via safe_int(..., None) if missing; no distinction between
truly-missing and user-entered null",
    "Time format flexibility: parser supports only decimal seconds (float); does NOT support
mm:ss or HH:MM:SS time strings (unlike some consuming tools), only raw float values"
],
"files": [
    "C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py",
    "C:/Users/avidu/Projects/sports-data-collector/src/cli/ingest.py",
    "C:/Users/avidu/Projects/sports-data-collector/src/core/db.py",
    "C:/Users/avidu/Projects/sports-data-collector/src/core/parsing.py",
    "C:/Users/avidu/Projects/sports-data-collector/src/core/validator.py",
    "C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/base.py",
    "C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/badminton.py",

```

```

"C:/Users/avidu/Projects/rally-annotator/docs/CSV_FORMAT.md"
]
},
{
  "subsystem": "Canonical Rally Schema & CSV I/O (sports-data-collector)",
  "summary": "The sports-data-collector defines a canonical rally record schema
(BadmintonRally/PickleballRally/TableTennisRally) with required fields video_id, rally_number,
start_time, end_time, and optional fields score_before/after, shots_count, ending_reason,
last_shot_outcome. Validation enforces non-negative start times, strictly positive durations,
chronological ordering, and no temporal overlaps. The canonical CSV (ROORA_BRIEF format)
includes 11 columns: rally_number, start_mmss, end_mmss, start_time, end_time, shots_count,
ending_reason, last_shot_outcome, score_before, score_after, notes. CVAT XML is converted to
this canonical format; import-local CSV ingest and export to eval sets both read/write canonical
columns only, dropping all other fields.",
  "key_facts": [
    "BadmintonRally schema: C:/Users/avidu/Projects/sports-data-
collector/src/core/schemas/badminton.py:4-22. Required: video_id (str), rally_number (int),
start_time (float), end_time (float). Optional: score_before/score_after (str), shots_count
(int), ending_reason (str), last_shot_outcome (str).",
    "Validation: C:/Users/avidu/Projects/sports-data-collector/src/core/validator.py:29-57
(validate_badminton_rallies). Rules: start_time >= 0, end_time > start_time
(Pydantic+validator), no temporal overlap between consecutive rallies, sorted by rally_number.",
    "Canonical CSV read: C:/Users/avidu/Projects/sports-data-
collector/src/parsers/cvat/rally_cvat.py:424-460 (load_canonical_csv). Reads decimal
start_time/end_time or mm:ss start_mmss/end_mmss; normalizes headers to lowercase; skips rows
missing rally_number or usable times; frame_min/frame_max set to -1 (CSV has no frame info).",
    "Canonical CSV write: C:/Users/avidu/Projects/sports-data-
collector/src/parsers/cvat/rally_cvat.py:356-372 (write_canonical_csv). 11 columns:
rally_number, start_mmss, end_mmss, start_time (%.3f), end_time (%.3f), shots_count,
ending_reason, last_shot_outcome, score_before, score_after, notes. Empty fields written as
blank strings.",
    "Canonical CSV columns (ROORA_BRIEF): C:/Users/avidu/Projects/sports-data-
collector/docs/annotation/ROORA_BRIEF.md:18-30. Enum ending_reason: 'winner' | 'forced_error' |
'unforced_error' | 'service_fault' | 'let' | 'other'. Enum last_shot_outcome: 'in' | 'net' |
'out' | 'n_a' (n_a for service_fault/let).",
    "Import-local CSV parse: C:/Users/avidu/Projects/sports-data-
collector/src/cli/curate.py:232-250. Reads CSV with DictReader, normalizes headers lowercase,
uses safe_int/safe_float for optional fields (defaults to None), required fields
(start_time/end_time) raise ValueError if missing. No sport/status/maturity/fractional columns
read; only 9 fields map to BadmintonRally.",
    "Export eval set: C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py:440-536.
Writes per-video CSV with 2 columns: start, end (decimal seconds only). No rally_number,
ending_reason, last_shot_outcome, shots_count, or scores in export; uses get_segment_intervals
(start_time, end_time only from database).",
    "CVAT conversion: C:/Users/avidu/Projects/sports-data-
collector/src/parsers/cvat/rally_cvat.py:325-353 (convert). Time recomputed from
frame_min/frame_max: start_time = frame_min / fps, end_time = frame_max / fps (vendor
start_time/end_time attributes discarded). Issues reported (not auto-fixed): zero_duration,
ending_reason_vocab, outcome_vocab, shot_density, frame_out_of_range, overlap,
possible_missed_rally.",
    "Issue detection allowed vocabularies: C:/Users/avidu/Projects/sports-data-
collector/src/parsers/cvat/rally_cvat.py:50-61. DEFAULT_ALLOWED_REASONS: {'winner',
'forced_error', 'unforced_error', 'service_fault', 'let', 'other'}. DEFAULT_ALLOWED_OUTCOMES:
{'in', 'net', 'out', 'n_a'}.",
    "Database schema: C:/Users/avidu/Projects/sports-data-collector/src/core/db.py:74-88.
badminton_rallies table: video_id (TEXT), rally_number (INTEGER), start_time (REAL), end_time
(REAL), score_before (TEXT), score_after (TEXT), shots_count (INTEGER), ending_reason (TEXT),
last_shot_outcome (TEXT). PK: (video_id, rally_number).",
    "Canonical column set on read (CSV / CVAT): rally_number, start_time (or start_mmss),
end_time (or end_mmss), shots_count, ending_reason, last_shot_outcome, score_before,
score_after, notes. Headers normalized to lowercase; missing optional fields degrade to None.",
    "Canonical column set on write (CSV export): ONLY start, end (decimal seconds). Rally
detail, ending_reason, shots_count, outcomes, scores, and video metadata NOT exported. Separate
manifest.json carries video_id, sport, num_segments, local_path, fps, resolution, license,
status."
  ]
}

```

```

"Canonical CSV in curate.py: C:/Users/avidu/Projects/sports-data-
collector/src/cli/curate.py:559 (write_canonical_csv arg). Called by convert-cvat command;
output CSV imports via import-local.",
"Vendor comparison: C:/Users/avidu/Projects/badminton-highlight-
indexer/backend/eval/eval_partial_labels.py (partial labels harness) and
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/calibrate_wasb.py:34-49
(load_golden_gts). Both read CSV with start_time, end_time columns ONLY; case-insensitive header
matching; no requirement for rally_number, ending_reason, shots_count, scores, or outcome
fields."
],
"contracts": [
    "BadmintonRally (& PickleballRally, TableTennisRally): video_id: str (required),
    rally_number: int (required, ?1, chronological), start_time: float (required, ?0), end_time:
    float (required, >start_time), score_before: str | None (default None), score_after: str | None
    (default None), shots_count: int | None (default None), ending_reason: str | None (default
    None), last_shot_outcome: str | None (default None). Pydantic validation: start_time ? 0;
    end_time > start_time.",
    "Canonical CSV columns (on read from CVAT/import-local): rally_number (int, required),
    start_time (float, required | start_mmss fallback), end_time (float, required | end_mmss
    fallback), shots_count (int | None), ending_reason (str | None, vocab:
    winner/forced_error/unforced_error/service_fault/let/other), last_shot_outcome (str | None,
    vocab: in/net/out/n_a), score_before (str | None, format: 'A-B'), score_after (str | None,
    format: 'A-B'), notes (str | None). Header normalization: all headers lowercased and stripped;
    field matching is lenient (aliases for last_shot_outcome: outcome, last_shot_outcome
    (in/net/out)).",
    "Canonical CSV columns (on write to export CSV): ONLY start (decimal seconds, float), end
    (decimal seconds, float). NO rally_number, ending_reason, shots_count, scores, or outcome.
    Written as 2-column CSV with header row.",
    "Canonical CSV columns (on write to CVAT-converted CSV): 11 columns in order:
    rally_number, start_mmss (format m:ss.mmm), end_mmss, start_time (format %.3f), end_time (format
    %.3f), shots_count (empty string if None), ending_reason (empty if None), last_shot_outcome
    (empty if None), score_before (empty if None), score_after (empty if None), notes (empty if
    None).",
    "Validation (validate_badminton_rallies): Returns (bool, List[str] errors). Checks:
    start_time ? 0, end_time > start_time, sorted chronologically by rally_number, no temporal
    overlap (end[i] <= start[i+1] for consecutive rallies). Blocker errors: timestamp violations,
    overlaps. Both CVAT and CSV paths feed this validator.",
    "CVAT issue detection (detect_issues): Returns List[Issue] with severity ('blocker' |
    'high' | 'medium' | 'low'), code (zero_duration, ending_reason_vocab, outcome_vocab,
    shot_density, frame_out_of_range, overlap, possible_missed_rally), message (human-readable),
    rally_number (optional). Blockers: zero_duration, overlap. NOT: sampling density (step/fps),
    since downstream padding absorbs ~0.5s uncertainty.",
    "Ending reason vocabulary (canonical enum): {'winner', 'forced_error', 'unforced_error',
    'service_fault', 'let', 'other'}. Shared across badminton, tennis, table tennis, pickleball,
    padel. Last-shot-outcome is SEPARATE and orthogonal (winner can have outcome='in'; error can
    have outcome='net'/'out'; service_fault/let have outcome='n_a').",
    "Import-local CSV format: flexible headers (case-insensitive), supports
    start_mmss/end_mmss OR start_time/end_time (mm:ss parsed via formula or direct float).
    rally_number defaults to 1-based row index if missing. shots_count, ending_reason,
    last_shot_outcome, score_before, score_after, notes are all optional (degrade to None). No
    sport/status/maturity/curation-level fields read; those come from CLI args."
],
"gaps_or_mismatches": [
    "ending_reason is PRESENT in canonical schema but NOT in eval CSV export (badminton-
    highlight-indexer loads only start_time/end_time from golden CSVs). Implication: the rally-
    indexer evaluation harness does NOT use ending_reason for scoring; it treats rallies as
    unlabeled intervals.",
    "last_shot_outcome is PRESENT in canonical schema but NOT in eval CSV export (same as
    ending_reason). The indexer calibration does not distinguish rally outcomes.",
    "shots_count is OPTIONAL in canonical schema and in CSV import but NOT captured in export
    CSVs. Database stores it; export drops it. Unclear if downstream uses this field.",
    "Fractional rally numbers (e.g., 1.1, 1.2 for replays/lets) are NOT supported by the
    canonical schema (rally_number is int). ROORA-BRIEF mandates integer 1-based rally_number with
    no gaps. If vendors annotate sub-rallies (e.g., 1st serve fault = 1, 1st let = 1.1, 2nd serve =
    1.2), the integer constraint will reject them or require manual splitting.",

```

"Curation status / maturity metadata (e.g., STAGING vs CURATED, confidence flags, rework needed) are NOT in the canonical CSV. They are stored in the database (videos.status enum: staging/curated/rejected) but do NOT round-trip through CSV export. Annotator/vendor CSVs have no analogous columns.",

"Missing end_time is NOT explicitly handled by the canonical schema. ROORA_BRIEF mandates end_time > start_time always. If a vendor omits end_time or provides end_time <= start_time, load_canonical_csv skips the row silently (no error logged). Import-local raises ValueError if end_time is missing/unparseable.",

"Annotation source / vendor / annotator ID is NOT in the canonical CSV. Video metadata carries match_name and video_id but no annotator/rework-status/version tracking per rally.",

"The canonical CSV is NARROWER on export than on import: export loses ending_reason, last_shot_outcome, shots_count, and scores. A CSV exported from the database and re-imported would preserve only start_time/end_time; all enrichment is lost. This is by design (eval set is intervals-only) but breaks round-trip fidelity.",

"CVAT attribute aliases are LENIENT (ending_reason, outcome, last_shot_outcome (in/net/out) all map to canonical last_shot_outcome), but CSV import/export do NOT tolerate variant column names. If a vendor-submitted CSV has 'outcome' instead of 'last_shot_outcome', it is silently ignored.",

"Frame-based metadata (frame_min, frame_max, n_boxes in CVAT RallyRecord) is NOT preserved in canonical CSV. Once converted to decimal seconds, frame info is lost. RallyRecord.load_canonical_csv sets frame_min/max to -1 as a marker."

```
],
"files": [
  "C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/badminton.py",
  "C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/base.py",
  "C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/pickleball.py",
  "C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/tabletennis.py",
  "C:/Users/avidu/Projects/sports-data-collector/src/core/validator.py",
  "C:/Users/avidu/Projects/sports-data-collector/src/core/parsing.py",
  "C:/Users/avidu/Projects/sports-data-collector/src/core/db.py",
  "C:/Users/avidu/Projects/sports-data-collector/src/parsers/cvat/rally_cvat.py",
  "C:/Users/avidu/Projects/sports-data-collector/src/parsers/badminton/coachai.py",
  "C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py",
  "C:/Users/avidu/Projects/sports-data-collector/docs/annotation/ROORA_BRIEF.md",
  "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/calibrate_wasb.py",
  "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/eval_partial_labels.py",
  "C:/Users/avidu/Projects/rally-annotator/docs/CSV_FORMAT.md",
  "C:/Users/avidu/Projects/rally-annotator/docs/ENDING_REASONS.md"
]
```

```
},
{
  "subsystem": "Sports Data Collector ? CVAT Vendor Ingestion & Pipeline Role",
  "summary": "The collector is the data producer in a two-repo training pipeline. It curates sports videos with per-rally temporal annotations [start, end) and commercial-license metadata, then exports to the badminton-highlight-indexer for processing and offline rally-segmenter training. The Roora vendor ingestion flow converts CVAT XML exports (or canonical CSVs) through a four-command pipeline: convert-cvat (normalizing frame?seconds), validate-delivery (QA gate with blocker checks + human review sheet), import-local (SQLite registration), and export (indexer-ready CSV+manifest). Manual self-rating via the VLC rally-annotator tool provides an alternative, lightweight ingestion path with minimal metadata. Golden-set role: temporal-only annotations for segmentation evaluation and training, explicitly not for frame-level shuttle detection; non-commercial sources excluded from default exports.",
```

```
  "key_facts": [
    "Collector role (PIPELINE_ROLE.md:1-46): data producer feeding badminton-highlight-indexer; goal is offline rally segmenter trained on commercially-licensed, annotated corpus to replace Gemini API",
```

```
    "Roora vendor pipeline (INGESTION_PIPELINE.md:7-44): four sequential commands: convert-cvat ? validate-delivery ? import-local ? export; converts CVAT XML boxes+attributes to canonical CSV with frame?seconds recomputation",
```

```
    "Critical frame-to-time conversion (rally_cvat.py:24-29): recomputes from frame_min/frame_max using true fps; vendor's start_time/end_time attributes explicitly ignored to avoid systematic errors from step-sampled jobs with wrong fps assumptions",
```

```
    "Canonical CSV schema (ROORA_BRIEF.md:14-30): vendor provides rally_number, start_mmss, end_mmss, shots_count, ending_reason, last_shot_outcome, score_before/after, notes; collector auto-fills start_time/end_time as decimal seconds",
```

"Validation rules (INGESTION_PIPELINE.md:68-92): blocker issues (overlaps, zero-duration) gate the import; high/medium issues (shot_density, possible_missed_rally, vocab) populate human review sheet; missed-rally detection via gap heuristic (>15s) and inflated-span flagging",

"Intake manifest fields (INTAKE_MANIFEST_TEMPLATE.csv:1-5): video_id, sport, format, fps (critical), duration_mmss, expected_rallies, assigned_to, status; fps provided to vendor for frame?seconds formula",

"VLC rally-annotator CSV (rally-annotator/README.md:10-14): minimal 5-column format (rally_number, start_time, end_time, ending_reason, sport) with decimal seconds only; no shots_count/scores; resumable (continues numbering on re-open)",

"Rally-annotator ingestion (CSV_FORMAT.md:26-41): direct import via curate import-local; optional fields populated as None; validation still runs (overlaps, positive duration); no external QA gate (self-produced)",

"Quality bar (ROORA_BRIEF.md:79?86): $\pm 0.3s$ boundary tolerance, zero missed rallies, schema compliance, shots_count exact for ?15 shots or ± 1 for longer rallies, 20% human spot-check on every delivery",

"Golden-set temporal-only contract (ADR-002, GOLDEN_SET_IMPLEMENTATION.md:6-8): (video_id, ordinal, start_time, end_time) shape shared across five sports; NOT used for shuttle detector training (lacks per-frame x,y labels); only for temporal segmenter training + eval",

"Regression testing (GS-2, GOLDEN_SET_IMPLEMENTATION.md:51-52): regress(video, tier, tolerance) compares predictions vs GT, flags when precision/recall drop >5% default; baseline snapshots per video with re-snapshot path for legitimate improvements",

"Non-commercial exclusion (INGESTION_PIPELINE.md:124-129): videos kept in DB regardless of license; default export commercial-only; non-commercial sources excluded unless --include-noncommercial flag passed",

"Observability reports (INGESTION_PIPELINE.md:131-144): per-video JSON (machine-readable), technical .md, and customer summary .md emitted on every validate-delivery/convert-cvat/import-local run for debugging/transparency"

],

"contracts": [

"Video identity contract: indexer keys by file MD5; collector by human video_id; manifest carries exact local_path so downstream processing runs on that exact file (acquisition precedes processing)",

"CSV schema contract: collector exports per-video (start, end) CSVs matching indexer's eval format; manifest.json pairs each CSV with video file; sport-agnostic (start_time, end_time) shape across badminton, tennis, cricket, pickleball, tabletennis",

"Canonical CSV format: rally_number (1-based, no gaps), start_time/end_time (decimal seconds), shots_count (optional int), ending_reason (enum: winner/forced_error/unforced_error/service_fault/let/other), last_shot_outcome (enum: in/net/out/n_a), score_before/after (optional 'A-B'), notes (required when ending_reason=other)",

"CVAT attribute aliases: lenient matching for rally metadata (e.g. 'score_before (a-b)' ? 'score_before'); frame-derived times override vendor-supplied times; frame range [frame_min, frame_max] at true fps = [start_time, end_time] seconds",

"Validation blockers: overlaps (end[N] > start[N+1]), zero-duration (end ? start), frame_out_of_range; violations exit non-zero and block import; non-blocker issues populate review sheet for human verification",

"Rally-annotator CSV format: rally_number, start_time (decimal), end_time (decimal), ending_reason, sport; optional columns may be added; consumers read by header name and skip rows where end ? start",

"Commercial license gate: is_commercial flag per video; default export includes only commercial sources; non-commercial explicitly excluded unless --include-noncommercial passed",

"Temporal-only annotation contract: (video_id, ordinal, start_time, end_time) shared across sports; no per-frame (x, y) shuttle coordinates; trained for segmentation (active vs. dead play), not detection; licensed/owned sources only"

],

"gaps_or_mismatches": [

"Partial-label strategy not explicitly documented: collector emphasizes 'zero missed rallies' as non-negotiable but offers no formal framework for handling genuinely ambiguous clips (warm-up detection, occluded play, administrative stoppages marked 'other')",

"Rally-annotator metadata subset: VLC tool omits shots_count, scores, notes ? fields that vendor includes; no documented pathway to enrich rally-annotator CSVs with post-hoc shot counts or scores before/after import",

"Missed-rally detection limitations acknowledged but not solved: gap heuristic (>15s) and shot_density flags are heuristic only; motion-based second-opinion stated as 'future enhancement' with no timeline",

"Quality bar vs. rally-annotator: vendor delivery expects 20% human spot-check; self-produced rally-annotator has no equivalent QA gate (rater is both producer and verifier, single POV)",

"Warm-up/knock-up exclusion rule: golden rule in ANNOTATION_GUIDE.md but no automated detection; rater must visually distinguish real play from pre-match cooperative rallying (relies on human judgment)",

"Frame sampling tolerance: documentation acknowledges ~0.5s sampling slack absorbed by clip padding but doesn't quantify impact on boundary accuracy for edge-case rallies",

"ADR-002 (golden set temporal-only) doesn't address: whether multi-annotator IAA (inter-annotator agreement) will be tracked; single-rater bias not factored into regression tolerance (ADR-004 reuses existing harness, may need sport-specific thresholds)"

```

],
"files": [
  "C:/Users/avidu/Projects/sports-data-collector/docs/PIPELINE_ROLE.md",
  "C:/Users/avidu/Projects/sports-data-collector/docs/INGESTION_PIPELINE.md",
  "C:/Users/avidu/Projects/sports-data-collector/src/parsers/cvat/rally_cvat.py",
  "C:/Users/avidu/Projects/sports-data-collector/docs/annotation/ROORA_BRIEF.md",
  "C:/Users/avidu/Projects/sports-data-collector/docs/annotation/ANNOTATION_GUIDE.md",
  "C:/Users/avidu/Projects/sports-data-collector/docs/annotation/INTAKE_MANIFEST_TEMPLATE.csv",
  "C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py",
  "C:/Users/avidu/Projects/rally-annotator/README.md",
  "C:/Users/avidu/Projects/rally-annotator/docs/CSV_FORMAT.md",
  "C:/Users/avidu/Projects/rally-annotator/docs/ENDING_REASONS.md",
  "C:/Users/avidu/Projects/badminton-highlight-indexer/docs/GOLDEN_SET_IMPLEMENTATION.md",
  "C:/Users/avidu/Projects/badminton-highlight-indexer/docs/archives/decisions/DECISIONS.md"
]
},
{
  "subsystem": "Manual Rally Annotation Artifacts",
  "summary": "Two distinct CSV formats exist for manual rally annotation: (1) the lean rally-annotator output with 5 core columns (rally_number, start_time, end_time, ending_reason, sport), and (2) the richer rally_review format with 14 columns adding metadata for manual curation (shot counts, time-in-minutes-seconds format, verification flags, reviewer comments). The ending_reason taxonomy is shared across both: winner, forced_error, unforced_error, service_fault, let, other. Manual ingesters must handle partial labeling (videos only partially tagged), fractional rally numbers (7.5, 16.5 from corrections), missing end_time values, status flags (VERIFIED/needs review), and free-text notes capturing net touchdowns and flagged anomalies like OVERLAP or INFLATED timing.",
  "key_facts": [
    "rally-annotator CSV columns (5): rally_number (int, 1-based monotonic), start_time (float, decimal seconds), end_time (float, decimal seconds, always > start_time), ending_reason (enum: winner|forced_error|unforced_error|service_fault|let|other), sport (enum: badminton|tennis|table_tennis|pickleball|padel) - CSV_FORMAT.md:6-18",
    "rally_review CSV columns (14): rally (int, rally identifier, may be fractional like 7.5 or 16.5), start_mmss (string, mm:ss.sss format, e.g. '1:02.062'), end_mmss (string, mm:ss.sss format), dur_s (float, duration in seconds), shots (int, shot count for the rally), ending_reason (enum or pipe-delimited alternates like 'forced_error|other'), ending_consistent (yes|NO, verification flag), notes (free-text, e.g. 'net', OVERLAP, INFLATED), auto_flag (status field for automated review), VERIFIED(Y/N) (Y/N or empty), true_start_mmss (optional corrected start), true_end_mmss (optional corrected end), true_ending (optional corrected reason), reviewer_comment (optional notes) - rally_review.csv header",
    "Ending reason taxonomy (6 values, shared across sports): winner (opponent couldn't return in-court shot), forced_error (loser missed under pressure), unforced_error (loser missed routine shot with time+position), service_fault (serve-rule violation), let (rally replayed under rules), other (occluded/administrative/unclear) - ENDING_REASONS.md:7-9",
    "rally_annotator.lua line 88 hard-codes header: 'rally_number,start_time,end_time,ending_reason,sport\\n'",
    "rally_annotator.lua line 262 writes CSV with %.3f precision for floats: 'd,%.3f,%.3f,%s,%s' (e.g. '1,37.434,72.194,winner,badminton')",
    "Actual lean CSV (Adarsh and Avi.rallies.csv): 40 rallies, rally_number 1-40 sequential, start_time range 37.434-599.712s, end_time range 72.194-604.962s, all have winning/error/service_fault labels, all sport=badminton",
    "Actual rich CSV (rally_review.csv): 26 rows, rally column shows integers AND fractional numbers (1,2,3... up to 26 but stored as rally NOT rally_number), times in mm:ss.sss format
  ]
}

```

(e.g. '0:03.003', '2:29.649'), shot counts range 1-15, ending_reason mostly single values but row 16 shows 'forced_error|other' (pipe-delimited alternatives), VERIFIED column has Y, NO, or empty, notes field captures qualitative flags ('net', 'OVERLAP', 'INFLATED')",

"Manual ingester realities: (1) PARTIAL LABELING - video may be only partially tagged, labels cover only a prefix/window of total runtime, (2) FRACTIONAL RALLY NUMBERS - corrections/insertions create numbers like 7.5 or 16.5, not just integers, (3) MISSING end_time - some fields may be blank (not shown in sample but possible), (4) STATUS FLAGS - VERIFIED(Y/N), ending_consistent(yes/NO), auto_flag field mark review state, (5) last_shot_outcome implicit in ending_reason enum, (6) shots_count as distinct column in review format, (7) FREE-TEXT NOTES - unstructured annotations like 'net touchdown' or anomaly flags ('INFLATED timing window', 'OVERLAP with prior rally')",

"rally_review CSV uses time-in-minutes-seconds string format (mm:ss.sss) while rally-annotator uses decimal-seconds floats (crucial for parsing)",

"rally-annotator v1.3 supports 5 sports: badminton, tennis, table_tennis, pickleball, padel (line 85-87 of Lua)",

"CSV parsing logic in rally_annotator.lua lines 222-250: forgiving parser that preserves extra columns, treats any leading-integer line as a rally row, skips header and blank lines"

],

"contracts": [

"LEAN ANNOTATOR CSV (5 columns): rally_number (int, 1-based, monotonic, continues across reopens), start_time (float, decimal seconds), end_time (float, decimal seconds, > start_time), ending_reason (enum: winner|forced_error|unforced_error|service_fault|let|other), sport (enum: badminton|tennis|table_tennis|pickleball|padel)",

"RICH REVIEW CSV (14 columns): rally (int or float, rally identifier, may be fractional), start_mmss (string, mm:ss.sss format), end_mmss (string, mm:ss.sss format), dur_s (float, seconds), shots (int, shot count), ending_reason (string, may be pipe-delimited alternatives like 'forced_error|other'), ending_consistent (enum: yes|NO), notes (string, free-text), auto_flag (string, status field), VERIFIED(Y/N) (enum: Y|N|empty), true_start_mmss (string or empty), true_end_mmss (string or empty), true_ending (string or empty), reviewer_comment (string or empty)",

"ENDING_REASON TAXONOMY (6 shared values across all sports): winner, forced_error, unforced_error, service_fault, let, other",

"SPORT TAXONOMY (5 supported): badminton, tennis, table_tennis, pickleball, padel",

"Decimal seconds format in lean CSV: floats with arbitrary precision but typically 3 decimal places (millisecond-level). Example: 37.434, 72.194",

"Time format in review CSV: string representation mm:ss.sss (minutes:seconds.milliseconds). Example: '0:03.003', '4:22.762'. Parser must convert to seconds for comparison with annotator output",

"Rally number uniqueness: within one video's CSV, rally_number is unique and monotonic. Fractional numbers (7.5, 16.5) may appear in review CSV when corrections create inserted records"

],

"gaps_or_mismatches": [

"TIME FORMAT MISMATCH: rally-annotator outputs decimal-seconds floats (37.434); rally_review uses mm:ss.sss string format (0:03.003). Ingester must convert review times to decimal-seconds or vice versa for reconciliation",

"COLUMN NAME MISMATCH: lean CSV has 'rally_number' (int counter), but rich CSV uses 'rally' (may be fractional). Manual ingester must handle both naming conventions and account for fractional IDs from edits/corrections",

"MISSING end_time: while both sample CSVs show complete (start, end) pairs, the schema allows blank end_time. Ingester must validate end > start and reject incomplete rallies",

"FRACTIONAL RALLY NUMBERS: rally-annotator logic assumes 1-based integer sequencing (line 283-286 of Lua), but real-world manual edits/insertions in review CSV introduce 7.5, 16.5-style identifiers. Ingester must parse floats, not just integers",

"ENDING_REASON ALTERNATIVES: lean CSV always single enum value; rich CSV row 16 shows 'forced_error|other' (pipe-delimited), indicating human uncertainty or net-touched ambiguity. Ingester must handle both scalar and pipe-delimited alternatives",

"OPTIONAL CORRECTION COLUMNS: rich CSV has true_start_mmss, true_end_mmss, true_ending columns (all empty in sample but semantically present). Ingester must treat these as optional overrides for the original times/reason if populated",

"PARTIAL LABELING: neither sample shows a video labeled only to 50% completion, but ENDING_REASONS.md and Lua code (lines 438-464) reference the pattern. Ingester must accept incomplete, prefix-window labels (e.g. rallies 1-15 out of 50 total)",

"STATUS FLAG SEMANTICS: VERIFIED and ending_consistent use inconsistent enum values (Y/N, yes/NO capitalization varies). Ingester must normalize or handle case-insensitively",


```

    "FREE-TEXT NOTES CONTENT: notes field captures sport-specific edge cases ('net touchdown'
in badminton, 'wall bounce' for padel) without schema. Ingestor must preserve as-is or define a
taxonomy if automated handling is needed",
    "AUTO_FLAG AND REVIEW STATE: auto_flag field (empty in sample) indicates automated quality
checks triggered (e.g. OVERLAP, INFLATED). Ingestor must define what flags are actionable vs.
informational"
],
"files": [
    "C:/Users/avidu/Projects/rally-annotator/docs/CSV_FORMAT.md",
    "C:/Users/avidu/Projects/rally-annotator/docs/ENDING_REASONS.md",
    "C:/Users/avidu/Projects/rally-annotator/vlc/rally_annotator.lua",
    "C:/Users/avidu/Projects/Annotation Setup/Collect/Adarsh and Avi.rallies.csv",
    "C:/Users/avidu/Projects/Annotation Setup/rally_review.csv"
]
},
{
    "subsystem": "Collector Output Contract for Rally Indexer Eval/Training",
    "summary": "The indexer consumes rally ground truth from the collector via a **manifest.json
+ per-video CSV** contract. The manifest (created by `sports-data-collector`'s `curate export`
command) pairs each video with a CSV of decimal-second rally intervals, plus metadata (fps,
resolution, video path, sport, license, curation status). The CSV columns are minimal:
`start,end` in seconds. The indexer's eval and training pipelines use **temporal IoU matching**
(greedy one-to-one at a threshold, default 0.5) to compare predicted rally windows against
ground truth; partial labels are supported only in the dedicated `eval_partial_labels.py`
harness via explicit `--window-start`/`--window-end` flags, not represented in the export format
itself.",
    "key_facts": [
        "calibrate_wasb.py:34-49 / load_golden_gts() ? reads CSV with columns (case-insensitive):
start_time, end_time (decimal seconds); returns sorted list of (start, end) tuples",
        "calibrate_local.py:66-74 / build_videos() ? expects manifest JSON with structure [{name,
trajectory, fps, frame_width, labels}, ...]; labels = path to golden CSV",
        "eval_partial_labels.py:47-54 / restrict_to_window() ? implements labeled-window concept:
filter predictions/GT to [window_start, window_end] interval; predicted segments outside window
are dropped (neither rewarded nor penalized)",
        "eval_partial_labels.py:68-78 ? window_end defaults to max(end_time) of all GT rallies;
window_start defaults to 0.0; both are CLI args not part of CSV/manifest",
        "rally_slicer.py:53-73 / load_rally_labels() ? loads CSV with columns (case-insensitive):
rally_number, start_time, end_time (decimal seconds OR mm:ss/HH:MM:SS); returns dict
rally_number ? (start, end) seconds",
        "distill_local.py:81-87 / load_video() ? reads manifest entry {fps, frame_width,
trajectory, labels}; calls load_golden_gts(labels) to get GT intervals; label_candidates()
matches candidate windows to GT via IoU",
        "training.py:50-62 / label_candidates() ? uses match_intervals() with iou_threshold
(default 0.5); candidate is positive iff it matches exactly one GT interval at IoU >=
threshold",
        "metrics.py:48-79 / match_intervals() ? greedy one-to-one matching: highest-IoU pairs
claimed first, each pred/GT used at most once; only pairs with IoU >= iou_threshold are
considered for matching",
        "metrics.py:15-29 / interval_iou() ? temporal intersection-over-union: overlap/(union)
both measured as durations in seconds; returns 0.0 for non-overlapping or degenerate intervals",
        "sports-data-collector curate.py:496-509 / export_eval_set() ? manifest entry structure:
{video_id, sport, csv (relative path), local_path, video_url, match_name, fps, resolution,
license_type, is_commercial, status, num_segments}",
        "sports-data-collector curate.py:490-494 ? exports CSV with header 'start,end' and rows
[start_seconds, end_seconds] (decimal, computed from DB intervals); one rally per row",
        "annotations.py:44-76 / load_annotations() ? legacy simple format: reads CSV with optional
'start'/'start_time' header row; columns 1-2 are start/end timestamps (decimal OR colon-
separated MM:SS/HH:MM:SS); blank lines and # comments ignored",
        "calibrate_local.py:19-20 ? manifest expected from collector at: `../sports-data-
collector/data/eval_export/manifest.json` (relative path hint in docs)",
        "batch.py:22-40 / resolve_video_path() ? normalizes manifest local_path: strips './',
converts backslashes to forward slashes, joins with videos_root if relative; cross-platform
handling for Windows drive letters (C:/...)",
        "eval_partial_labels.py:72-82 ? labeled-window enforcement: load_golden_gts() gets all
rallies; manually compute/override window_end from CLI; restrict_to_window() filters both

```

```

predictions and GT to in-window region before scoring",
    "distill_local.py:48-69 / build_candidates() ? proposal stage (velocity_thresh=0.005,
merge_gap=1.0, min_dur=0.5) generates many candidates; WASB trajectory parses trajectory.csv
with columns Frame, Visibility, X, Y"
],
"contracts": [
    "CSV (ground truth): columns start,end | format: decimal seconds (float) | one rally per
row | optional header 'start' or 'start_time' | blank/# lines ignored | no other columns
required/consumed",
    "CSV (human annotations, collector intake): columns rally_number, start_time, end_time,
ending_reason, sport [optional: start_mmss, end_mmss, shots_count, score_before, score_after,
last_shot_outcome, notes] | start/end can be decimal seconds OR MM:SS / HH:MM:SS | rally_number
= string (supports inserted IDs like '7.5') | indexer ignores all columns except
start_time/end_time",
    "manifest.json structure: {commercial_only, statuses, total_videos, total_segments,
eval_sets: [{video_id, sport, csv (relative path to start,end CSV), local_path, video_url,
match_name, fps (float), resolution (string or null), license_type, is_commercial (bool),
status, num_segments}, ...]}",
    "Trajectory CSV (WASB input): columns Frame, Visibility, X, Y | parsed by
TrackNetRunner.parse_trajectory_csv() | motion data required only for calibration/distillation,
not for evaluating labels alone",
    "IoU matching contract: temporal intervals (start, end) in seconds | IoU >= threshold ?
match candidate | greedy one-to-one ? TP counted | threshold defaults to 0.5, configurable via
--iou flag",
    "Labeled-window concept (partial labels): window_start, window_end (seconds) | predicted
intervals outside [window_start, window_end) are dropped before scoring | GT intervals assumed
already within window | NOT part of manifest/CSV; applied only in eval_partial_labels.py via CLI
args",
    "Metadata contract (in manifest): fps (float, required for trajectory windowing and
feature extraction) | frame_width (float, required for WASB velocity normalization) | local_path
(required, string, file system path; used as key with file MD5 for cross-repo identification) |
resolution (optional, string like '1080p')",
    "Feature extraction for training: candidate window bounds ? {duration, peak_velocity,
mean_velocity, active_frame_count (YOLO) OR active_ratio (WASB), track_id_count (YOLO) OR
net_crossings (WASB)} | features must be fps/resolution-INVARIANT except where explicitly
normalized by frame_width"
],
"gaps_or_mismatches": [
    "Labeled-window restriction (eval_partial_labels.py concept) is NOT representable in the
current collector export format ? it is enforced only as CLI args to eval_partial_labels.py, not
persisted in manifest/CSV. If collector needs to support partial labels at scale, a new optional
manifest field (e.g., window_start, window_end) would be required.",
    "Rally metadata (ending_reason, last_shot_outcome, score_before/after, shots_count) are
collected and validated by the collector but NOT consumed by indexer eval/training pipelines ?
they are discarded after validation. Only (start_time, end_time) flow downstream.",
    "Trajectory CSV path is NOT in the manifest ? it is supplied separately as a CLI arg
(--trajectory) to calibrate_wasb, calibrate_local, eval_partial_labels, and distill_local. The
distill pipeline requires it to build features but there is no contract guaranteeing it exists
or is co-located.",
    "Resolution field in manifest is optional (nullable) and rarely used downstream ? fps is
required but resolution is mainly for observability. Feature extraction normalizes by
frame_width instead of resolution string.",
    "Partial-label support assumes GT labels cover a PREFIX of the video only (every rally up
to window_end is labeled, everything after is unlabeled). This assumption is NOT validated or
documented in the manifest ? it is left to the user to enforce via --window-end.",
    "CSV header detection (start, start_time) is case-insensitive but relies on exact column
name matching (no fuzzy matching) ? a vendor sending 'start_seconds' or 'start_frame' would fail
silently or cause the harness to read the wrong column.",
    "No contract for multi-sport data in a single manifest ? eval_sets can contain mixed
sports but most eval/training scripts assume homogeneous sport (e.g., training.label_candidates
does not filter by sport, so mixing sports in one training set would train a sport-agnostic
model).",
    "IoU threshold is a global setting (--iou flag) applied uniformly across all videos in a
manifest ? no per-video or per-sport threshold override supported in the current contract."
],

```

```

"files": [
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\calibrate_wasb.py",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\calibrate_local.py",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\eval_partial_labels.py",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\tools\\rally_slicer.py",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\distill_local.py",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\training.py",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\metrics.py",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\annotations.py",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\batch.py",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\run_phase3_eval.py",
    "C:\\Users\\avidu\\Projects\\sports-data-collector\\src\\cli\\curate.py"
],
{
    "subsystem": "Sports Data Collector - DB/Provenance & Observability Layer",
    "summary": "The collector stores video metadata and sport-specific annotations (rallies, points, deliveries) in SQLite with two-level schema: a central `videos` table holding provenance (source URL, license, sport, video_id) and curation status (STAGING/CURATED/REJECTED), plus sport-specific child tables (badminton_rallies, tennis_points, cricket_deliveries, etc.) containing segment metadata. Observability is layered via obsreport (soft dependency) which generates per-video reports at import/validation stages, capturing parse/validation/registration metrics and flags for delivery issues; local imports bypass vendor-issued flags but still emit reports showing license, commerciality, and segment counts at registration.",
    "key_facts": [
        "C:/Users/avidu/Projects/sports-data-collector/src/core/db.py:52-71: videos table schema ? video_id (PK), sport, license_type (enum: MIT/Apache/BSD/CC0/CC-BY/CC-BY-NC/CC-BY-NC-SA/Public-Domain/Proprietary/Unknown), license_url, is_commercial (boolean flag), status (enum: staging/curated/rejected), match_name, fps, resolution; foreign keys ensure on-delete cascade to child tables.",
        "C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/base.py:24-27: CurationStatus enum ? STAGING, CURATED, REJECTED; used for video-level workflow state only, no segment-level maturity.",
        "C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/base.py:12-22: LicenseType enum ? includes Proprietary (self-rated), MIT/Apache/BSD/CC0/CC-BY (vendor-delivered), CC-BY-NC/CC-BY-NC-SA (non-commercial), PUBLIC_DOMAIN, UNKNOWN; no source tracking (self-rated vs vendor-delivered not distinguished).",
        "C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py:109-122: import_local sets all local imports to CURATED status by default (line 117: 'Custom local imports bypass staging by default'), skipping vendor staging workflow.",
        "C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py:73-78: License commercial-safety check via validator.check_license_commercial(license_type) ? whitelist includes Proprietary (line 21 in validator.py), so self-rated Proprietary is treated as commercial-safe.",
        "C:/Users/avidu/Projects/sports-data-collector/src/reporting/adapters.py:113-156: build_import_recorder() ? observability envelope for import-local; parse stage logs records_parsed; validation stage reports is_valid (boolean); registration stage captures sport, license (value string), is_commercial, status (value string); no annotation author/timestamp/maturity.",
        "C:/Users/avidu/Projects/sports-data-collector/src/reporting/spec.yaml:25-47: Spec rules emit two flags at import: license_noncommercial (warn if non-commercial), missing_fps (warn if fps null). Rules only check video-level fields; no segment-level flags for imports.",
        "C:/Users/avidu/Projects/sports-data-collector/src/core/db.py:210-237: insert_badminton_rallies() ? clears existing rallies for video_id then bulk-inserts new ones; no versioning, audit trail, or per-segment metadata beyond sport schema (rally_number, start_time, end_time, score, shots_count, ending_reason, last_shot_outcome).",
        "C:/Users/avidu/Projects/sports-data-collector/src/reporting/__init__.py:48-77: emit_import_report() ? called from curate.py:145-154 after DB insert; optional (soft dependency); writes next to video or to fallback_dir if video absent; always succeeds (never raises).",
    ]
}

```

```

"C:/Users/avidu/Projects/sports-data-collector/src/parsers/cvat/rally_cvat.py: Issue class
? severity (blocker/high/medium/low), code, message, optional rally_number; only delivery
validation emits Issue objects; import-local does not detect or report Issues.",
"C:/Users/avidu/Projects/sports-data-collector/src/core/validator.py:29-144:
validate_*_rallies() ? checks temporal non-overlap and positive duration; no quality metrics,
annotator tracking, or maturity scores.",
"C:/Users/avidu/Projects/sports-data-collector/tests/test_reporting.py:83-112: import
observability envelope captures only is_valid (boolean), errors (list of strings), count
(segments), sport, source_format; no flags for import-local validation issues."
],
"contracts": [
  "VideoMetadata schema: video_id, sport(enum), video_url?, local_path?, license_type(enum),
  license_url?, is_commercial(bool), status(CurationStatus enum: staging/curated/rejected),
  match_name, fps?, resolution?",
  "CurationStatus enum: STAGING | CURATED | REJECTED ? video-level only, no segment maturity
  states",
  "LicenseType enum: MIT, Apache-2.0, BSD, CC0, CC-BY, CC-BY-NC, CC-BY-NC-SA, Public-Domain,
  Proprietary, Unknown ? no source marker (self-rated vs vendor-delivered)",
  "is_commercial: boolean derived from license_type whitelist check; whitelist in
  config.yaml includes Proprietary (defaults to [MIT, Apache-2.0, BSD, CC0, CC-BY, Public-Domain,
  Proprietary])",
  "Import observability envelope (obsreport.RunRecorder): stages=(parse, validation,
  registration); parse.metrics.records_parsed; validation.status(ok/error),
  validation.metrics.valid(bool); registration.metrics.segments_registered(int),
  registration.details={sport, license(str), is_commercial(bool), status(str)}",
  "Delivery observability envelope: stages=(parse, validation);
  parse.metrics.rallies_parsed; validation.metrics.issues_blocker/high/medium/low; flags carry
  code, severity(error/warn/info), message, stage, customer_message?, evidence={rally_number?,
  rally_frames?, fps?}",
  "Spec rules: license_noncommercial (warn if is_commercial==false), missing_fps (warn if
  fps==null); rules trigger only at import/delivery validation, never at re-tagging",
  "RallyRecord (CVAT adapter): rally_number, start_time, end_time, shots_count?,
  ending_reason?, last_shot_outcome?, score_before?, score_after?, notes?, frame_min, frame_max,
  n_boxes, vendor_start_time?, vendor_end_time?",
  "Database cascade: video DELETE triggers cascade on child tables (badminton_rallies,
  tennis_points, cricket_deliveries, pickleball_rallies, tabletennis_rallies); child tables have
  no independent delete cascade"
],
"gaps_or_mismatches": [
  "No annotation source tracking: LicenseType enum includes Proprietary but does not
  distinguish self-rated (manual local import) from vendor-delivered (CVAT/external parser);
  license provenance assumes external source unless registered via import-local, but no explicit
  marker.",
  "No segment-level maturity: segments (rallies/points/deliveries) lack status/maturity
  fields; video-level STAGING/CURATED/REJECTED does not map to individual segment review state
  (needs_review -> confirmed -> gold not trackable).",
  "No annotator/author metadata: neither VideoMetadata nor rally schemas capture who
  created/reviewed annotations; no timestamp for import or re-tagging.",
  "Local import skips staging workflow: import_local (line 117 in curate.py) hardcodes
  CURATED status, bypassing vendor review pipeline; observability report does not flag this
  deviation.",
  "Import validation does not emit observability flags:
  DatasetValidator.validate_*_rallies() only check temporal integrity (overlaps, duration);
  emit_import_report() logs errors only as string list, not structured Flags; no per-segment QA
  metrics.",
  "No re-tagging audit trail: insert_badminton_rallies() (line 215-216 in db.py) DELETE-
  then-INSERT erases history when re-importing annotations for the same video_id; no versioning or
  conflict detection.",
  "No label quality regression tracking: no metrics for comparing old vs. new annotations
  when re-tagging (e.g., shot_count changes, ending_reason divergence); no drift detection.",
  "License whitelist hardcoded in config fallback: check_license_commercial()
  (validator.py:21) defaults to fixed list including Proprietary; no per-video or per-sport
  override; no rationale/audit trail for why Proprietary is commercial-safe.",
  "Observability rules only video-level: spec.yaml:25-47 defines only video-scope rules
  (license_noncommercial, missing_fps); no rules for segment defects at import time (gaps,

```

```

outliers, schema violations).",
    "No import-local annotation Issues: CVAT delivery parser emits Issue objects
(blocker/high/medium/low) but import-local does not detect or report Issues to observability
envelope; local vs. vendor validation asymmetry."
],
"files": [
    "C:/Users/avidu/Projects/sports-data-collector/src/core/db.py",
    "C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/base.py",
    "C:/Users/avidu/Projects/sports-data-collector/src/core/validator.py",
    "C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py",
    "C:/Users/avidu/Projects/sports-data-collector/src/cli/ingest.py",
    "C:/Users/avidu/Projects/sports-data-collector/src/reporting/__init__.py",
    "C:/Users/avidu/Projects/sports-data-collector/src/reporting/adapters.py",
    "C:/Users/avidu/Projects/sports-data-collector/src/reporting/spec.yaml",
    "C:/Users/avidu/Projects/sports-data-collector/src/parsers/cvat/rally_cvat.py",
    "C:/Users/avidu/Projects/sports-data-collector/tests/test_reporting.py",
    "C:/Users/avidu/Projects/sports-data-collector/tests/test_validator.py",
    "C:/Users/avidu/Projects/sports-data-collector/tests/test_local_import.py"
]
}
]

```

2. assistant (2026-06-08T08:43:43.697Z)

I'll analyze the findings to propose positioning the collector around the label lifecycle. Let me verify key claims against the actual source files before finalizing recommendations.

3. user (2026-06-08T08:43:44.680Z)

```

1      from pydantic import BaseModel, Field
2      from typing import Optional
3      from enum import Enum
4
5      class SportType(str, Enum):
6          BADMINTON = "badminton"
7          TENNIS = "tennis"
8          TABLE_TENNIS = "tabletennis"
9          PICKLEBALL = "pickleball"
10         CRICKET = "cricket"
11
12         class LicenseType(str, Enum):
13             MIT = "MIT"
14             APACHE_2 = "Apache-2.0"
15             BSD = "BSD"
16             CC0 = "CC0"
17             CC_BY = "CC-BY"
18             CC_BY_NC = "CC-BY-NC"
19             CC_BY_NC_SA = "CC-BY-NC-SA"
20             PUBLIC_DOMAIN = "Public-Domain"
21             PROPRIETARY = "Proprietary"
22             UNKNOWN = "Unknown"
23
24         class CurationStatus(str, Enum):
25             STAGING = "staging"
26             CURATED = "curated"
27             REJECTED = "rejected"
28
29         class VideoMetadata(BaseModel):
30             video_id: str = Field(..., description="Unique alphanumeric identifier for the
video")
31             sport: SportType
32             video_url: Optional[str] = Field(None, description="Remote URL of the video (if
available)")
33             local_path: Optional[str] = Field(None, description="Local path to video file (if
downloaded or custom)")

```

```

34     license_type: LicenseType
35     license_url: Optional[str] = None
36     is_commercial: bool = Field(False, description="Flag indicating if the license
allows commercial use")
37     status: CurationStatus = CurationStatus.STAGING
38     match_name: str = Field(..., description="Name of the match/event")
39     fps: Optional[float] = Field(None, description="Frames per second")
40     resolution: Optional[str] = Field(None, description="Resolution string (e.g. 1080p,
720p)")
41

```

4. user (2026-06-08T08:43:45.272Z)

```

440     def export_eval_set(self, args):
441         """Export curated annotations as indexer-ready eval/training sets.
442
443         Emits, per qualifying video, a `

```

```

492         writer.writerow(["start", "end"])
493     for start, end in intervals:
494         writer.writerow([start, end])
495
496     manifest.append({
497         "video_id": video.video_id,
498         "sport": sport.value,
499         "csv": os.path.relpath(csv_path, out_dir).replace(os.sep, "/"),
500         "local_path": video.local_path,
501         "video_url": video.video_url,
502         "match_name": video.match_name,
503         "fps": video.fps,
504         "resolution": video.resolution,
505         "license_type": video.license_type.value,
506         "is_commercial": video.is_commercial,
507         "status": video.status.value,
508         "num_segments": len(intervals),
509     })
510
511     os.makedirs(out_dir, exist_ok=True)
512     manifest_doc = {
513         "commercial_only": commercial_only,
514         "statuses": [s.value for s in statuses],
515         "total_videos": len(manifest),
516         "total_segments": sum(m["num_segments"] for m in manifest),
517         "eval_sets": manifest,
518     }
519     manifest_path = os.path.join(out_dir, "manifest.json")
520     with open(manifest_path, "w", encoding="utf-8") as f:
521         json.dump(manifest_doc, f, indent=2)
522
523     print(f"\n=== Exported eval/training set to '{out_dir}' ===")
524     if manifest:
525         summary = [[m["sport"], m["video_id"], m["num_segments"],
526                     "YES" if m["is_commercial"] else "NO",
527                     os.path.basename(m["local_path"]) if m["local_path"] else "(no
local file)"]]
528         for m in manifest]
529         print(tabulate(summary, headers=["Sport", "Video ID", "Segments", "Comm?",
"Video file"], tablefmt="grid"))
530         print(f"\nVideos exported : {len(manifest)}")
531         print(f"Total segments : {manifest_doc['total_segments']}")
532         if skipped_noncomm:
533             print(f"Skipped (non-commercial) : {skipped_noncomm} (pass --include-
noncommercial to include)")
534         if skipped_empty:
535             print(f"Skipped (no annotations) : {skipped_empty}")
536         print(f"Manifest : {manifest_path}\n")
537
538     def convert_cvat(self, args):
539         """Convert a vendor CVAT rally export into our canonical per-rally CSV.
540
541         Absorbs the CVAT bounding-box delivery format (rally fields carried as box
542         attributes) on the ingestion side: each rally's time is recomputed from
543         its frame range at the video's true fps, and content defects (overlaps,
544         zero-duration, off-vocabulary endings, coarse sampling) are written to an
545         issues report rather than silently "fixed". The emitted CSV imports via
546         `import-local`; any blocker issue must be resolved first, since the
547         importer's validator rejects overlapping / zero-duration rallies.
548         """
549         from src.parsers.cvat.rally_cvat import (
550             convert, write_canonical_csv, render_issue_report,
551         )
552         try:
553             result = convert(args.input, fps=args.fps,

```

```

554             video_path=args.video_path, step=args.step)
555     except (FileNotFoundError, ValueError) as e:
556         print(f"Error: {e}")
557         sys.exit(1)
558
559     write_canonical_csv(result.records, args.out)

```

5. assistant (2026-06-08T08:43:50.477Z)

Now let me verify the partial-labels harness and the regression/golden-set flow that the lens hinges on.

6. user (2026-06-08T08:43:50.705Z)

```

40     Config = Tuple[float, float, float]
41
42     # Operating points to compare. The tuned config wins on its fit-video but over-segments
43     # and does not transfer; baseline (+ net-crossing gate) is the stronger general point.
44     TUNED: Config = (0.05, 1.0, 1.0)
45
46
47     def restrict_to_window(intervals: List[Interval], w0: float, w1: float) ->
List[Interval]:
48         """Keep intervals that overlap the labeled window [w0, w1].
49
50         A predicted segment is evaluated iff it overlaps the labeled region; segments wholly
51         in the un-labeled tail (start >= w1) or before the window (end <= w0) are dropped so
52         they are neither rewarded nor penalized. GTs are assumed already inside the window.
53         """
54         return [(s, e) for (s, e) in intervals if e > w0 and s < w1]
55
56
57     def _eval_point(predict_fn: Callable[[Config], List[Interval]], cfg: Config, points: List[Interval], fps: float, gts: List[Interval],
58                     window: Tuple[float, float], gate_crossings: int, iou: float) -> dict:
59         """Score one (config, gate) operating point inside the labeled window."""
60         preds = predict_fn(cfg)
61         if gate_crossings > 0 and points:
62             preds = rally_gate.apply_gate(points, preds, fps, min_crossings=gate_crossings)
63             preds = restrict_to_window(preds, *window)
64             sc = score(preds, gts, iou)
65             return {"cfg": cfg, "gate": gate_crossings, "n_pred": len(preds), "score": sc,
"preds": preds}
66
67
68     def run(trajecory_csv: str, labels_csv: str, fps: float, frame_width: float,
69            window_start: float = 0.0, window_end: Optional[float] = None,
70            gate_crossings: int = 2, iou: float = 0.5,
71            comparison_out: Optional[str] = None) -> dict:
72         gts_all = load_golden_gts(labels_csv)
73         if not gts_all:
74             raise SystemExit(f"no usable human rallies in {labels_csv}")
75         if window_end is None:
76             window_end = max(e for _, e in gts_all)
77         window = (window_start, window_end)
78         gts = restrict_to_window(gts_all, *window)
79
80         predict_fn = make_predict_fn(trajecory_csv, fps, frame_width)
81         points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
82         traj_end = (max(p.frame for p in points) / fps) if points else 0.0
83
84         print("=== Pipeline vs PARTIAL human labels ===")
85         print(f"labels: {labels_csv}")
86         print(f" {len(gts_all)} human rallies; labeled window [{window_start:.1f}s,
{window_end:.1f}s] "
87               f"-> {len(gts)} rallies in-window")

```



```
88         print(f"trajectory covers ~0-{{traj_end:.1f}}s "  
89               f"({{'covers the window' if traj_end >= window_end - 1 else 'WARNING: shorter  
than window'}})")
```

7. user (2026-06-08T08:43:50.836Z)

```
52     def _init_db(self):  
53         with self._get_connection() as conn:  
54             cursor = conn.cursor()  
55  
56             # 1. Create common videos table  
57             cursor.execute("""  
58             CREATE TABLE IF NOT EXISTS videos (  
59                 video_id TEXT PRIMARY KEY,  
60                 sport TEXT NOT NULL,  
61                 video_url TEXT,  
62                 local_path TEXT,  
63                 license_type TEXT NOT NULL,  
64                 license_url TEXT,  
65                 is_commercial INTEGER NOT NULL DEFAULT 0,  
66                 status TEXT NOT NULL DEFAULT 'staging',  
67                 match_name TEXT NOT NULL,  
68                 fps REAL,  
69                 resolution TEXT  
70             )  
71             """)  
72  
73             # 2. Create badminton_rallies table  
74             cursor.execute("""  
75             CREATE TABLE IF NOT EXISTS badminton_rallies (  
76                 video_id TEXT NOT NULL,  
77                 rally_number INTEGER NOT NULL,  
78                 start_time REAL NOT NULL,  
79                 end_time REAL NOT NULL,  
80                 score_before TEXT,  
81                 score_after TEXT,  
82                 shots_count INTEGER,  
83                 ending_reason TEXT,  
84                 last_shot_outcome TEXT,  
85                 PRIMARY KEY (video_id, rally_number),  
86                 FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE  
87             )  
88             """)  
89  
90             # 3. Create tennis_points table  
91             cursor.execute("""
```

8. assistant (2026-06-08T08:43:55.945Z)

The findings are well-corroborated by the source. The key gaps the lens hinges on are confirmed: (1) `CurationStatus` is video-level only (no `needs\_review` -> confirmed -> gold` segment maturity), (2) the labeled-window concept lives only as `eval_partial_labels.py` CLI args (not in the manifest), (3) no provenance/annotator/source marker on `videos`, and (4) `insert_badminton_rallies()` is DELETE-then-INSERT with no re-tag audit trail. Let me verify the golden-set regression flow claim.

9. user (2026-06-08T08:43:57.063Z)

```
1     # Golden-Set Subproject ? Design & Implementation Overview  
2  
3     **Status:** In progress · **Date:** 2026-05-30  
4     **Companion docs:** design rationale + vendor shortlist in  
[GOLDEN_SET_VENDORING.md](GOLDEN_SET_VENDORING.md) (ADR-006); scoring mechanics in  
[EVALUATION.md](EVALUATION.md).
```

```

5
6     This is the **implementation tracker** for turning the golden-set/vendor *design* into
shipped code. It's split into small, independently-reviewable PRs. The whole subproject is
**vendor-agnostic by construction** ? only one slice (GS-3, `HumanVendorProvider`) depends on
which vendor you pick offline; everything else can land first.
7
8     > **Update (2026-06-02):** Vendor selected for the **initial pilot ? Roora**
(Bengaluru). The **annotation guideline, intake-manifest template, and NDA** are authored in the
collector repo `docs/annotation/`, and the collector's `import-local` ingestion now covers the
**3 pilot sports** (badminton, pickleball, table tennis). On the indexer side, GS-1
(`file_provider.py`), GS-2 (`backend/eval/regression.py` + `run_regression.py`), and the GS-4
oracle scaffold (`automated_provider.py`) have landed; the remaining slice is the vendor-
dependent **GS-3 `HumanVendorProvider`**, parked until the Roora pilot confirms quality.
9
10    ---
11
12    ## Architecture in one picture
13
14    ```
15                video_ref ??? AnnotationProvider.submit()/poll()/fetch() ???
List[(start,end)]  ground truth
16                ?           ?           ?           ?
17                FileProvider HumanVendor Automated(oracle) ?
18                (GS-1)       (GS-3)       (GS-4)
backend.eval.metrics.score() ??? existing, unchanged
19
20    production pipeline ??? DB ??? backend.eval.harness.get_predictions() ???
List[(start,end)]  predictions
21
22                                regress(video, tier) (GS-2)
compares the two
23    ```
24
25    **The keystone:** a provider's `fetch()` returns the **same `List[(start, end)]`** that
`eval.annotations.load_annotations()` already returns and `eval.metrics.score()` already
consumes. So one scorer grades all three of: product-vs-GT, vendor-vs-answer-key, and annotator-
vs-annotator (IAA). No new scoring code ? only new *sources* of intervals.
26
27    ---
28
29    ## PR breakdown
30
31    | Slice | Branch | Scope | Depends on | Vendor-dependent? |
32    |---|---|---|---|---|
33    | **GS-1** | `feat/annotation-provider-registry` | `AnnotationProvider` ABC + registry +
`FileProvider` + tests | ? | No |
34    | **GS-2** | `feat/regression-runner` | `regress(video, tier, tolerance)` over existing
harness + `baseline.json` snapshot/compare + CLI | GS-1 | No |
35    | **GS-3** | `feat/human-vendor-provider` | `HumanVendorProvider` (submit/poll/fetch
against the chosen vendor's API or file-drop), DPA/secure-link handling | GS-1, **your vendor
pick** | **Yes** |
36    | **GS-4** | `feat/automated-oracle-provider` | `AutomatedProvider` fast-tier oracle ?
**interface stub** (injected labeler + fake; concrete model = backlog) | GS-1, GS-2 | No |
37    | **GS-5** | `feat/regression-triggers` | Wire triggers: CI gate, on-demand CLI,
scheduled run | GS-2, GS-4 | No |
38
39    Each row is one PR. GS-1 ? GS-2 ? GS-4 ? GS-5 can proceed **now**, independent of vendor
selection. GS-3 is parked until you've chosen a vendor (then it's a thin adapter behind the GS-1
interface).
40
41    ---
42
43    ## Slice details
44
45    ### GS-1 ? Provider registry + FileProvider ? (this PR)

```

```

46     - `backend/annotations/base.py` ? `AnnotationProvider` ABC (`submit`/`poll`/`fetch` +
`annotate()` convenience) and `annotation_registry`, mirroring `backend/providers` and
`backend/indexer` registry patterns.
47     - `backend/annotations/file_provider.py` ? `FileProvider`: the "job" is a CSV that
already exists (formalizes today's manual/ShuttleSet flow); reuses
`eval.annotations.load_annotations`.
48     - Tests assert the registry contract **and** the keystone (provider output is directly
scorable by `metrics.score()`).
49     - **Design choice:** `fetch()`/`annotate()` raise loudly on a missing/incomplete job
rather than returning `[]`, so a broken label source can't silently masquerade as "0 rallies"
and skew a regression to green.
50
51     ### GS-2 ? Regression runner (next)
52     `regress(video, tier="fast", tolerance=0.05)`: get GT from
`annotation_registry.get(tier)`, read predictions via the existing `eval.harness`, score with
the existing `metrics`, compare against a per-video `baseline.json`, and flag a regression when
precision/recall drop beyond tolerance. Pure orchestration over existing parts + a small
baseline store. CLI: `python run_regression.py --video ... --tier file`.
53
54     ### GS-3 ? HumanVendorProvider (after your vendor pick)
55     A concrete provider that submits a (licensed/owned-only) video to the chosen vendor and
fetches the returned rally CSV. Carries the privacy controls from the design doc (secure time-
limited links, delete-on-delivery, DPA). Thin adapter ? the interface already exists from GS-1.
56
57     ### GS-4 ? AutomatedProvider (fast-tier oracle) ?? oracle rule
58     An independent strong labeler for the fast/CI tier. **Must be a different model lineage
than whatever production uses** (else shared blind spots ? false green). A fast-tier failure is
a *smoke alarm* that escalates to the GS-3 human tier, never an authoritative verdict on its
own.
59
60     ### GS-5 ? Regression triggers

```

10. assistant (2026-06-08T08:44:04.991Z)

I now have full confirmation. The golden-set regression flow lives on the indexer side (`backend/eval/regression.py`, `backend/annotations/`), keyed by `tier`, with the keystone being `List[(start,end)]` intervals ? and the collector is the **source** that feeds GTs through the `FileProvider`/manifest path. The critical insight for the lens: the collector's `status` enum and DELETE-then-INSERT have no notion of label maturity or re-tag tracking, so the golden-set regression flow cannot distinguish a `confirmed` from a `gold` label, nor know how much of a video is labeled.

I have everything needed to ground the recommendations. Submitting the structured output.

11. user (2026-06-08T08:45:25.448Z)

Structured output provided successfully

12. assistant (2026-06-08T08:45:36.594Z)

I've completed the analysis and submitted the structured output. Here is a summary of the label-lifecycle / provenance product lens I proposed for the collector.

****Core repositioning:**** The collector's real product is the LABEL LIFECYCLE ? it should be the system of record for WHAT is trustworthy (provenance + license + maturity per source) and HOW MUCH of each video is labeled (the labeled-window). Both facts are already needed downstream but are not captured.

****Grounding evidence (verified in source):****

- `CurationStatus` is only `STAGING|CURATED|REJECTED` (base.py:24-27) ? a workflow state, not a `needs\_review` -> confirmed -> gold` maturity ladder.
- The labeled-window is fully implemented on the indexer (`eval\_partial\_labels.py:47-78`) but lives only as CLI args, defaulting `window\_end` to `max(end\_time)` ? wrong when a video is tagged past its last rally.

- `videos` table (db.py:52-71) and `VideoMetadata` carry no provenance/annotator/timestamp ? self-rated imports and QA-gated vendor deliveries land identically.
- `import\_local` hardcodes `status=CURATED` (curate.py:109-122), bypassing review.
- `insert\_badminton\_rallies()` is DELETE-then-INSERT (db.py:210-237) ? re-tagging erases history, defeating the golden-set regress flow's ability to tell improvement from drift.
- The golden-set regression flow (GOLDEN_SET_IMPLEMENTATION.md GS-1/GS-2) grades everything off the same `List[(start,end)]` the collector exports, so the collector decides which intervals are gold.

Six recommendations, two P0 (maturity ladder; persist labeled-window in DB + manifest), two P1 (provenance/annotator/timestamp; versioned re-tag audit feeding regression), two P2 (structured QA flags on import-local; carry the new fields through the manifest contract).

The key design constraint I flagged: enrichment belongs in the **manifest**, not the eval CSV ? the CSV stays intervals-only, and `ending\_reason`/`last\_shot\_outcome`/`shots\_count` remain DB-only and unused downstream by design.